

POS Lernaufgabe - Projektverwaltung Advanced REST API

Bearbeite die Aufgaben selbstständig. Die Angabe ist bewusst ähnlich knapp wie eine Prüfungsangabe: Entscheide selbst, welche DTOs, Mapper, Service-Methoden und Statuscodes sinnvoll sind.

Technische Vorgaben

- Verwende Minimal API oder Controller.
- Verwende kein Repository-Pattern.
- LINQ-Abfragen befinden sich im Service.
- Das Service darf direkt mit dem ProjectsContext arbeiten.
- DTOs und Mapper dürfen verwendet werden.
- Gib keine vollständigen Entity-Objekte zurück.
- Verwende sinnvolle HTTP-Statuscodes.
- Validiere Eingaben einfach, aber nachvollziehbar.
- Die bestehende Projektstruktur soll beibehalten werden.

Aufgabe 1: Projekt-Dashboard

Erstelle einen Endpunkt, der eine kompakte Übersicht über Projekte liefert.

```
GET /api/projects/dashboard?status=Active&customerCountry=AT&skill=CSharp
```

Die Antwort soll pro Projekt ungefähr enthalten:

- Projektname
- Kunde
- Status
- Deadline
- Anzahl aller Tasks
- Anzahl offener Tasks
- Anzahl überfälliger Tasks
- Anzahl beteiligter Mitarbeiter
- verbrauchtes Budget anhand der TimeEntries
- benötigte Skills

Alle Query-Parameter sind optional. Die Filterung soll im Service mit LINQ erfolgen.

Schwierigkeit: Include, ThenInclude, Any, Count, Sum, verschachtelte Projektion.

Aufgabe 2: Mitarbeiter-Workload

Erstelle einen Endpunkt, der die Auslastung von Mitarbeitenden berechnet.

```
GET /api/employees/workload?department=Development&country=AT&onlyActive=true&minHours=20
```

Die Antwort soll pro Mitarbeiter ungefähr enthalten:

- Name
- Abteilung
- Rolle

- Land des Büros
- Anzahl zugewiesener offener Tasks
- Anzahl aktiver Projekte
- gebuchte Stunden
- durchschnittliche Stunden pro Task
- Skills des Mitarbeiters

Die Parameter sind optional. `onlyActive=true` soll nur aktive Mitarbeiter berücksichtigen. `minHours` soll nach gebuchten Stunden filtern.

Schwierigkeit: GroupBy, Sum, Average, Filter auf Owned Type Office.Country.

Aufgabe 3: Kritische Tasks

Erstelle einen Endpunkt, der kritische oder problematische Tasks findet.

```
GET /api/tasks/critical?dueBefore=2026-12-31&priority=High&missingSkillsOnly=true&take=10
```

Die Antwort soll Tasks liefern, die z. B. offen, blockiert, überfällig oder kritisch priorisiert sind.

Die Antwort soll ungefähr enthalten:

- Tasktitel
- Projektname
- Kunde
- Priorität
- Status
- Fälligkeitsdatum
- zugewiesene Person
- benötigte Skills
- Skills, die der zugewiesenen Person fehlen

`missingSkillsOnly=true` soll nur Tasks liefern, bei denen die zugewiesene Person nicht alle benötigten Skills besitzt.

Schwierigkeit: Vergleich zweier Many-to-many-Listen, Any, All, Contains, optionale Filter.

Aufgabe 4: Kundenanalyse

Erstelle einen Endpunkt, der Kunden nach Projektaktivität analysiert.

```
GET /api/customers/analysis?country=AT&industry=Finance&minProjects=2
```

Die Antwort soll pro Kunde ungefähr enthalten:

- Kundenname
- Branche
- Land
- Anzahl Projekte
- Anzahl aktiver Projekte
- gesamtes Projektbudget
- tatsächlich gebuchte Kosten
- Anzahl offener Tasks über alle Projekte

- nächster Projekttermin

Alle Parameter sind optional.

Schwierigkeit: Aggregation über mehrere Navigationsebenen: Customer -> Projects -> Tasks -> TimeEntries.

Aufgabe 5: Skill-Gap-Report

Erstelle einen Endpunkt, der zeigt, welche Skills in Projekten gebraucht werden und ob genug Mitarbeitende dafür vorhanden sind.

```
GET /api/skills/gap-report?category=Backend&minRequiredTasks=2
```

Die Antwort soll pro Skill ungefähr enthalten:

- Skillname
- Code
- Kategorie
- Anzahl Tasks, die diesen Skill benötigen
- Anzahl Mitarbeiter, die diesen Skill besitzen
- Projekte, in denen der Skill benötigt wird
- ob ein Mangel besteht

Ein Skill gilt z. B. als Mangel, wenn er in mehreren offenen Tasks benötigt wird, aber nur wenige aktive Mitarbeitende ihn besitzen.

Schwierigkeit: Many-to-many in beide Richtungen, SelectMany, Distinct, GroupBy, berechnete Werte.

Zusatzanforderungen

- Baue für mindestens 2 Endpunkte Paging ein.
- Sortiere Ergebnisse sinnvoll.
- Begrenze pageSize, z. B. maximal 50.
- Verwende BadRequest, wenn Pflichtparameter fehlen oder Werte ungültig sind.
- Verwende NotFound, wenn ein konkreter Datensatz per ID nicht existiert.
- Schreibe die LINQ-Abfragen möglichst so, dass sie von EF Core in SQL übersetzt werden können.
- Keine komplette Entity-Rückgabe, sondern eigene DTOs.

Optional Extra Schwer

Erstelle zusätzlich einen vollständigen Report für ein einzelnes Projekt.

```
GET /api/projects/{id}/full-report
```

Der Report soll ein einzelnes Projekt mit Kunde, Tasks, Mitarbeitern, Skills, TimeEntries, Budgetverbrauch und offenen Risiken zurückgeben.